



University of Minho
School of Engineering

Relatório do projeto

Simulação de uma clínica de saúde

Cláudia Teixeira a110414

Fernando Costa a110808

Docentes: José Carlos Filipe Ramalho

Luís Filipe Costa Cunha

6 de janeiro de 2026

Índice

1	Introdução	1
1.1	Contexto	1
1.2	Objetivo	1
1.3	Processo criativo inicial	2
2	Funcionamento do sistema	3
2.1	Enquadramento básico	3
2.2	Parâmetros da Simulação	6
3	Ficheiros base	7
3.1	Chegadas.py	7
3.2	Triagem.py	8
3.3	Consultorios.py	18
4	Simulação de eventos	31
4.1	Lógica de tempos e eventos	31
4.2	Evento de Chegada	33
4.3	Evento de Saída da Triagem	36
4.4	Evento de Saída do Consultório	37
4.5	Resultados e Obtenção de Dados	38
5	Interface e Modelo Visual	39
5.1	Menu	39
5.2	Ajuda	40
5.3	Configurações	41
5.4	Iniciar Simulação	44
5.4.1	Balcões	45
5.4.2	Cadeiras	47
5.4.3	Consultórios	47
5.4.4	Botões de tempo de espera	49
5.4.5	Tempo	50

6	Resultados e Representações Gráficas	53
6.1	Tamanho das Filas VS Tempo	53
6.2	Ocupação dos Médicos VS Tempo	55
6.3	Tamanho das Filas de Espera VS Taxas de Chegada	56
6.4	Restante Informação	57
7	Conclusões	57

1 Introdução

1.1 Contexto

As estimativas dos tempos e tamanhos das filas de espera de estabelecimentos de saúde é um dos temas e preocupações principais quando o tema é eficácia de gestão de recursos. Um programa de simulação de chegadas de pacientes a uma clínica é um exemplo de uma forma de tentar perceber quais estratégias podem agilizar o atendimento de pacientes, garantindo a divisão de tarefas equitativas entre órgãos diferentes da clínica.

1.2 Objetivo

Ao elaborar uma simulação deste género, é esperado obter resultados relativos a:

1. Intervalos de tempo de:
 - 1.1 filas de espera;
 - 1.2 atendimento;
 - 1.3 ocupação de recursos.
2. Tamanhos de filas de espera;
3. Taxas de:
 - 3.1 número de pacientes atendidos;
 - 3.2 número de pacientes por atender;
 - 3.3 ocupação de recursos em função do número de pacientes chegados à clínica.

Com o tratamento destes dados, podemos inferir a eficácia e a exigência dos vários recursos em função de vários parâmetros, como número de chegadas de pacientes, quantidade de recursos em serviço e tempo.

O objetivo da elaboração deste projeto, no âmbito da unidade curricular de **Algoritmos e Técnicas de Programação**, é implementar uma simulação de eventos discretos que modela o funcionamento de uma clínica de saúde, permitindo avaliar o impacto de diferentes parâmetros no desempenho do sistema.

1.3 Processo criativo inicial

Aquando da disponibilização do guião do projeto, foi possível reconhecer de imediato a sua dimensão e complexidade face ao nível de conhecimentos de programação até então adquiridos. Ainda assim, a ambição de desenvolver um projeto bem estruturado e o desafio associado ao desconhecido motivaram o início do trabalho.

Após a análise das instruções iniciais, iniciou-se um processo de reflexão e definição conceitual do projeto. Desde a fase inicial, destacou-se a intenção de criar uma clínica composta por diferentes unidades médicas, correspondentes a diferentes especialidades.

Adicionalmente, procurou-se aproximar o modelo da realidade, introduzindo estatutos de prioridade que permitissem um atendimento mais eficaz a pessoas com algum tipo de incapacidade. Com o mesmo objetivo de realismo, foi também definido um processo de triagem prévio à consulta, representando o funcionamento típico da receção de uma clínica.

Ainda numa fase preliminar, estabeleceu-se que a clínica funcionaria durante 12 horas diárias, no período compreendido entre as 9:00 e as 21:00. Com estes pressupostos definidos, iniciou-se a fase de criação de código.

2 Funcionamento do sistema

2.1 Enquadramento básico

O programa é um exemplo de uma **simulação de eventos discretos** em que uma lista de **eventos** define os acontecimentos da simulação. O programa foi elaborado em **Python**, com o auxílio de bibliotecas, como **FreeSimpleGUI**, **numpy** e **matplotlib**.

Os pacientes que participam nesta simulação são carregados de uma **base de dados** de um ficheiro **JSON**, e adicionados a uma **lista de dicionários**, sendo que todas as informações relevantes relativas ao paciente, quer sejam informações da base de dados, quer informações e **parâmetros** calculados ao longo da simulação.

Considerando a disponibilização de uma base de dados contendo informação relativa aos pacientes, optou-se pela sua utilização. No entanto, originalmente cada doente era caracterizado por parâmetros que não se encaixam propriamente no contexto do projeto, alguns sendo desnecessários. Assim sendo, foram removidas as características irrelevantes e adicionados outros que seriam úteis mais à frente:

```
{
    "id": "p0",
    "BI": "91702023-5",
    "nome": "Neyanne Sampaio",
    "idade": 47,
    "sexo": "feminino",
    "incapacidade": false,
    "bebe_ao_colo": false,
    "gravidez": false
}
```

A cada paciente foram removidas algumas **chaves** que não seriam úteis no contexto da simulação e adicionadas: “incapacidade”, “bebe ao colo”, “gravidez”, cujo valor é um **boolean**. Estas características seriam fundamentais para mais tarde definir **prioridades**. Para além da remoção, foram adicionadas estas chaves com algumas condições para fazer com que a base de dados se aproximasse da realidade: Pessoas com menos de 15 anos e mais de 50 anos

não poderiam estar grávidas; Pessoas com menos de 15 anos não poderiam estar com bebé ao colo; Da base de dados total, somente cerca de 20 por cento das pessoas apresentam pelo menos um **critério de prioridade**.

Após o estabelecimento das condições sobre os dados a tratar, iniciou-se a **arquitetura do sistema** para o funcionamento da clínica.

Da lista, apenas alguns darão entrada na clínica em função do **tempo de simulação**, ou seja, são carregados e é-lhes atribuído um **tempo de chegada**. Enquanto os tempos de chegada atribuídos forem menores que o **tempo final de simulação**, mais doentes darão entrada na clínica. A partir do momento em que os tempos de chegada são superiores ao tempo final de simulação, mais nenhum doente dará entrada na clínica.

Os intervalos entre os tempos de chegada dos pacientes seguem uma **distribuição exponencial negativa** com aproximação a um **parâmetro de Poisson**. Assim que dão entrada na clínica, os pacientes são encaminhados para uma **fila de espera para a triagem**, a qual pode ser uma **fila prioritária** ou uma **fila sem prioridade**, ou seja, a fila de espera para a triagem é composta por uma **lista de duas listas**.

Para além disso, os pacientes que tenham uma **consulta marcada** têm prioridade na fila não prioritária em relação a quem não tem consulta; assim, sempre que um doente com consulta marcada entra na clínica e segue para a fila de triagem, ocupa a posição atrás do último paciente da fila sem consulta marcada, e à frente do primeiro paciente sem consulta marcada.

Da fila de triagem, os pacientes são atendidos em **balcões de atendimento**, e, tal como na fila de triagem, podem ser **prioritários ou não prioritários**. Quando um balcão fica disponível, o primeiro doente da lista respetiva sai da fila e ocupa o balcão. No balcão, o **tempo de atendimento** segue uma **distribuição normal**, sendo o tempo de entrada na triagem igual ao tempo de simulação em que o paciente ocupa o balcão, e o tempo de saída da triagem igual ao tempo de entrada acrescido do tempo de triagem. Estes valores são registados no **dicionário do doente**.

Depois dos balcões de atendimento, os pacientes seguem para as **filas de espera para os consultórios**, onde são separados em filas segundo **especialidades**. Estas filas encontram-se igualmente divididas em filas com consulta marcada e filas sem consulta marcada, mantendo-se a prioridade dos pacientes com consulta.

Para modelar as filas de consultas, utiliza-se um **dicionário**, cujas chaves correspondem às especialidades e cujos valores são listas de pacientes com e sem consulta, representados pelos respetivos dicionários.

A fila de espera para os consultórios encaminha, em primeiro lugar, para as **secções** divididas por especialidades; dentro de cada secção existe um **dicionário de consultórios**, sendo cada consultório definido pelo seu **id** e pelo respetivo **médico**.

Os médicos, à semelhança dos pacientes, são carregados a partir de um **ficheiro JSON** e organizados num **dicionário**, no qual todos os parâmetros e critérios — quer provenientes da base de dados, quer gerados ao longo da simulação — são registados, como por exemplo o início e fim de turno, o número de doentes atendidos e o **estado de disponibilidade**.

Quando um consultório de uma secção fica disponível, o primeiro paciente da fila com consulta ocupa o consultório; caso esta fila esteja vazia, o primeiro paciente da fila sem consulta é atendido. Os tempos de consulta seguem uma **distribuição normal**, sendo a **média do tempo de consulta** variável consoante a **especialidade médica**. Assim que um paciente termina a consulta, o seu estado é registado como "**ATENDIDO**", e todos os dados recolhidos durante a simulação são utilizados para o cálculo das **métricas de desempenho**.

No final, é possível extrair resultados relativos a **tempos médios de espera**, **tempos de consulta e atendimento**, **número de pacientes atendidos em balcões e consultórios** e **taxa de ocupação dos recursos**.

2.2 Parâmetros da Simulação

Parâmetros	Definição
Doentes	Carregados a partir de um ficheiro JSON
Médicos	Carregados a partir de um ficheiro JSON
Taxas de Chegada	• Seguem uma distribuição normal; • Com aproximação a um parâmetro de Poisson; • Variam ao longo do dia; • Modos alteráveis na interface.
Número de balcões	Parâmetro alterável na interface
Tempo de Triagem	Segue uma distribuição normal
Tempo de Consulta	Segue uma distribuição normal
Prioridade	Definida na base de dados (condições do doente)
Ter Consulta	Variável aleatória, definida por uma função random
Especialidade	Variável aleatória, definida por uma função random

É importante mencionar que a compreensão destes parâmetros exige a interpretação global do projeto ou do relatório.

3 Ficheiros base

3.1 Chegadas.py

Inicialmente para as chegadas, foi criada uma função para gerar chegadas de acordo com taxas geradas.

```
tfinal = 12*60

def gera_tempos_chegada(tfinal):
    #taxas = [(início do bloco, fim do bloco, doentes que chegam/hora)]

    taxas = [
        (0, 2 * 60, 18 / 60),
        (2 * 60, 4 * 60, 11 / 60),
        (4 * 60, 7 * 60, 7 / 60),
        (7 * 60, 9 * 60, 9 / 60),
        (9 * 60, 12 * 60, 16 / 60)
    ]
    tchegadas = []
    tempo_atual = 0.0

    for hinicio, hfim, taxa in taxas:
        #começa no início do bloco ou no tempo atual
        tempo_atual = max(tempo_atual, hinicio)
        #gera chegadas dentro do bloco
        while tempo_atual < hfim and tempo_atual < tfinal:
            intervalo = np.random.exponential(1 / taxa)
            tempo_atual += intervalo

        if tempo_atual < hfim and tempo_atual < tfinal:
            tchegadas.append(round(tempo_atual,2))
    return tchegadas
```

A partir desta função, foi possível gerar todas as chegadas de pacientes antecipadamente, antes do início da simulação. Com base nas taxas definidas, foi implementada uma distribuição de Poisson não homogénea, em que a taxa de chegada de pacientes varia ao longo do dia. Registam-se maiores intensidades nos períodos de pico: no início do dia (das 9:00 às 11:00, correspondendo às primeiras duas horas de funcionamento da clínica, com taxa de chegada definida como 18/60) e no final da tarde (das 18:00 às 21:00, correspondendo às últimas três horas antes do fecho, com taxa de chegada definida como 16/60).

Os tempos entre chegadas são gerados segundo uma distribuição exponencial, cuja média é inversamente proporcional à taxa de chegada, garantindo, desta forma, a caracterização de um processo de Poisson não homogéneo ao longo do dia definido por `intervalo = np.random.exponential(1 / taxa)`.

Optou-se por não utilizar a função de pré-geração dos tempos de chegada, pois concluiu-se que a geração das chegadas em tempo real durante a simulação assegura um comportamento mais consistente do sistema, impedindo esperas artificiais e permitindo uma integração mais natural com a evolução temporal da simulação e a interface. Assim sendo, as chegadas são geradas ao longo da própria simulação e a função anteriormente mencionada foi descartada. [Ver explicação da geração de chegadas dos doentes.](#)

O método de geração de chegadas utilizado no programa é tratado durante a simulação, que será abordado numa secção mais à frente.

3.2 Triagem.py

Agora que os pacientes já chegam à clínica e recebem um tempo de chegada, **precisam de ser atendidos na triagem**. Este passo é importante porque é quando os parâmetros de prioridade inicialmente atribuídos são usados para organizar os doentes na fila. Num novo ficheiro “**Triagem.py**” foi criada uma função:

```
def Doentes(ficheiro):  
    doentes = []  
    f = open(ficheiro, "r", encoding = "utf-8")  
    lista = json.load(f)  
    for pessoa in lista:
```

```

doente = {
    "nome": pessoa["nome"],
    "idade": pessoa["idade"],
    "sexo": pessoa["sexo"],
    "id": pessoa["id"],
    "BI": pessoa["BI"],
    "consulta": temConsulta(),
    "especialidade": doenca(),
    "prioridade": prioridadeIndividual(pessoa),
    "tchegada": None, #Chegada ao posto médico
    "tentrada_triagem":None, #Entrada na triagem
    "tsaida_triagem": None, #Saída da triagem
    "tentrada_consultorio" : None, #Entrada no consultório,
    "tsaida_consultorio":None, #Saída do consultório
    "aguarda_consultorio" :False,
    "estado_final" : None
    #que representa o estado em que cada doente está ‘preso’ quando a simulação a
    #(t_atual == 12*60) (FILA_TRIAGEM, TRIAGEM, FILA_CONSULTORIO, CONSULTA, ATENDI
}

doentes.append(doente)

f.close()

return doentes

```

Através desta função, o doente, já carregado da base de dados, ganha agora ainda mais atributos. De todos estes, são relevantes mencionar agora **“consulta”**, **“especialidade”** e **“prioridade”**:

Sendo temConsulta():

```

def temConsulta():
    #para perceber se o doente tem consulta ou não
    #(consideramos que 80% das pessoas tem consulta)
    consulta = False
    n = random.randint(1,100)

```

```
if n <= 80:
    consulta = True
return consulta
```

A função temConsulta serve para diferenciar os pacientes que têm consulta marcada dos que não têm consulta porque na fila para a triagem serão organizados também por este estatuto.

Sendo doenca():

```
def doenca():
    n = random.randint(1,101)
    if n <= 7:                                     #7% 1 médico
        doenca = "Dermatologia"

    elif n > 7 and n<= 13:                         #+6% = 13% 1 médico
        doenca = "Gastroenterologia"

    elif n> 13 and n <= 18:                       #+5% = 18% 1 médico
        doenca = "Pneumonologia"

    elif n> 18 and n <= 26:                       #+8% = 26% 2 médico
        doenca = "Cardiologia"

    elif n>26 and n <= 29:                         #+3% = 29% 1 médico
        doenca = "Endocrinologia"

    elif n>29 and n <= 38:                       #+9% = 38% 2 médicos
        doenca = "Ortopedia"

    elif n>38 and n <= 44:                       #+6% = 44% 1 médico
        doenca = "Neurologia"
```

```
elif n>44 and n <= 54:          #+10% = 54% 2 médicos
    doenca = "Ginecologia e Obstetrícia"

elif n>54 and n <= 58:          #+4% = 58% 1 médico
    doenca = "Psiquiatria"

elif n>58 and n <= 70:          #+12% = 70% 2 médicos
    doenca = "Pediatria"

else:                            #+30% = 100% 3 médicos
    doenca = "Medicina Geral"

return doenca
```

Esta função é fundamental porque define as proporções de cada especialidade, ou seja, o número de doentes com certa especialidade vai estar distribuído diferentemente. Por exemplo, um paciente tem uma maior probabilidade de apresentar um problema ou consulta de Medicina Geral (30 por cento) do que de Endocrinologia (3 por cento). Esta função também é um grande auxílio para mais tarde gerar a base de dados com os médicos da clínica. Assunto abordado mais à frente. A função gera um número aleatório entre 1 e 100 e cada especialidade tem um intervalo de números que pode ser maior ou menor consoante a “popularidade” da mesma.

Sendo `prioridadeIndividual(pessoa)`:

```
def prioridadeIndividual(pessoa):
    prior = False
    if pessoa["incapacidade"] == True or pessoa["bebe_ao_colo"] == True or
    pessoa["gravidez"] == True or pessoa["idade"] <= 2 or pessoa["idade"] >= 75:
        prior = True
    return prior
```

A função de prioridade individual somente lê a base de dados e se a pessoa tiver alguma incapacidade (“incapacidade” : True), um bebé ao colo (“bebé ao colo” : True), estiver

grávida ("gravidez" : True) ou ter mais de 75 anos de idade, é considerada uma pessoa com estatuto prioritário ("prioridade": True)

Após a atribuição destes parâmetros, os doentes são organizados na fila para a triagem com base nos mesmos pelo que, os doentes com **estatuto prioritário ficam em primeiro na lista**, de seguida ficam os não prioritários com consulta e, por fim, os não prioritários sem consulta. Esta organização é possível através da função `chegadaAntesTriagem` presente no ficheiro "Triagem.py".

```
def chegadaAntesTriagem(doente, fila_triagem):
#Fila antes de ser atendido na receção, ou seja, à chegada à clínica
# doente prioritário fica atrás do último prioritário da fila
    prioritarios=fila_triagem[0]
    resto= fila_triagem[1]
#o resto são os pacientes que não são prioritários
if doente["prioridade"] == True:
    prioritarios.append(doente)

    elif doente["consulta"] == True :
        j = 0
        while j < len(resto) and resto[j]["consulta"]:
            j += 1
        resto.insert(j, doente)
    else:
        resto.append(doente)
        #quem não tem consulta, vai para o fim da lista de espera inteira

    return fila_triagem
```

Nesta função, os doentes prioritários são colocados atrás do último doente prioritário já existente na fila, garantindo que pessoas com maior necessidade de atenção são atendidas primeiro. Entre os restantes, os doentes com consulta estão posicionados à frente dos que não têm consulta, mantendo a ordem relativa dos pacientes com consulta. Os doentes sem consulta são adicionados ao final da fila.

A triagem em si é feita nos balcões de atendimento que estão definidos por:

```
balcoes = [{balcão}]
```

```
balcão = {"id": int , "prioritario": bool, "disponivel" : bool, "doente" : dic,  
"entrada" : float, "saida" : float, "ndoentes_atendidos" : int,  
"tempo_ocupado" : float}
```

```
balcoes = [  
{"id":1, "prioritario":True, "disponivel":True, "doente":None, "entrada":None,  
"saida":None, "ndoentes_atendidos" : 0, "tempo_ocupado": 0.0},  
{"id":2, "prioritario":False, "disponivel":True, "doente":None, "entrada":None,  
"saida":None, "ndoentes_atendidos" : 0, "tempo_ocupado": 0.0},  
{"id":3, "prioritario":False, "disponivel":True, "doente":None, "entrada":None,  
"saida":None, "ndoentes_atendidos" : 0, "tempo_ocupado": 0.0}]
```

Através da função `criaBalcoes` cada balcão regista se está disponível, o doente atendido, o tempo de entrada e saída, o número de doentes atendidos e o tempo total de ocupação.

```
def criaBalcoes(config_atual):  
    nbalcoes = config_atual["nbalcoes"]  
    nbalcoes_prior = config_atual["nbalcoes_prior"]  
    balcoes = []  
    for i in range(1, nbalcoes + 1):  
        balcoes.append({  
            "id": i,  
            "prioritario": False,  
            "disponivel": True,  
            "doente": None,  
            "entrada": None,  
            "saida": None,  
            "ndoentes_atendidos": 0,  
            "tempo_ocupado": 0.0  
        })
```

```
#Marca os primeiros nbalcoes_prior como prioritários
for i in range(0,nbalcoes_prior):
    balcoes[i]["prioritario"] = True

return balcoes
```

Agora que os balcões estão criados podem ser ocupados. Mas ainda não há nada que defina o tempo de triagem, por isso mesmo é que a função `tempoTriagem` foi criada:

```
def tempoTriagem(t_atual):
    t_Triagem = max(0.5,np.random.normal(loc=3.0, scale=1.0))
    #distribuição normal para tempo na triagem que em média
    #demora 3 minutos e desvio padrão de 1 minuto
    entrada_triagem = t_atual
    saida_triagem = t_atual + t_Triagem
    t_atual += t_Triagem
    return entrada_triagem, saida_triagem, t_Triagem
```

Esta função usufrui da **distribuição normal** para gerar tempos de triagem que tendem para 3 minutos e podem sofrer um desvio-padrão de 1 minuto, garantindo variabilidade realista nos tempos de atendimento. Aqui também são registados os tempos que permitem atualizar o dicionário do doente e o balcão, garantindo variabilidade realista nos tempos de atendimento.

Durante a simulação, a função `ocupar_balcaoTriagem` verifica quais balcões estão disponíveis e atribui doentes da fila, atualizando os tempos de entrada e saída de cada paciente, bem como o estado do balcão:

```
def ocupar_balcaoTriagem(lista,balcoes,t_atual):
    fila_prioritario = lista[0]
    fila_resto=lista[1]
    eventos_gerados=[]
```

```
for balcao in balcoes:
    #se o balcão for prioritário e estiver livre
    if balcao["prioritario"] and balcao["disponivel"] and fila_prioritario:
        doente = fila_prioritario.pop(0) #remove o primeiro paciente da fila
        entrada = max(t_atual, doente["tchegada"])
        #o max define o tempo de início da triagem como o maior
        #entre t_atual e tchegada:
        #se o balcão ficar livre antes da chegada do doente, usa tchegada;
        #se estiver livre depois da chegada, usa t_atual

        _, saida, t_Triagem = tempoTriagem(entrada) #atualiza os tempos

        balcao.update({
            "doente": doente,
            "disponivel": False,
            "entrada": entrada,
            "saida": saida,
            "ndoentes_atendidos" : balcao["ndoentes_atendidos"] + 1,
            "tempo_ocupado": balcao["tempo_ocupado"] + t_Triagem
        })

    #também conseguimos atualizar os dados do paciente
    doente.update({
        "tentrada_triagem" : entrada,
        "tsaida_triagem": round(saida,2),
        "estado_final": "TRIAGEM"
    })

    #e ainda, geramos um evento de saída de triagem
    evento = {
        "tempo" : doente["tsaida_triagem"],
        "tipo" : "SAI_TRIAGEM",
```

```

        "doente" : doente,
        "balcao" : balcao["id"]
    }
    eventos_gerados.append(evento)

#se o balcão não for prioritário e estiver livre
elif not balcao["prioritario"] and balcao["disponivel"] and fila_resto:
    doente = fila_resto.pop(0)
    entrada = max(t_atual, doente["tchegada"])
    _, saida, t_Triagem = tempoTriagem(entrada)

    balcao.update({
        "doente": doente,
        "disponivel": False,
        "entrada": entrada,
        "saida": saida,
        "ndoentes_atendidos" : balcao["ndoentes_atendidos"] + 1,
        "tempo_ocupado": balcao["tempo_ocupado"] + t_Triagem
    })

    doente.update({
        "tentrada_triagem" : entrada,
        "tsaida_triagem": round(saida,2),
        "estado_final" : "TRIAGEM"
    })

    evento = {
        "tempo" : doente["tsaida_triagem"],
        "tipo" : "SAI_TRIAGEM",
        "doente" : doente,
        "balcao" : balcao["id"]
    }

```

```
eventos_gerados.append(evento)
```

```
return lista, balcoes, eventos_gerados
```

Após o atendimento ao balcão, os doentes são removidos do balcão e o seu estado é atualizado para “FILA CONSULTORIO” no ficheiro de simulação que será explorado na secção 4., indicando que aguardam agora para consulta. Para que isto aconteça é necessário haver uma função que atualize o estado do balcão para livre novamente. A função que permite isto é desocupar balcaoTriagem:

```
def desocupar_balcaoTriagem(balcoes, t_atual):
    eventos_gerados = []
    evento = None
    for balcao in balcoes:
        if not balcao["disponivel"] and t_atual >= balcao["saida"]:

            doente = balcao["doente"]

            balcao.update({
                "doente": None,
                "disponivel": True,
                "entrada": None,
                "saida": None
            })

            doente["estado_final"] = "FILA_CONSULTORIO" #registro

            eventos_gerados.append(evento)
    return balcoes, eventos_gerados
```

Em resumo, os eventos do tipo SAI TRIAGEM são gerados no momento em que um doente inicia a triagem, pois é nesse instante que se conhece o tempo de conclusão do serviço.

A função `desocupar balcaoTriagem` não gera novos eventos, limitando-se a atualizar o estado do sistema (libertação do balcão e transição do doente para a fila de consulta). Esta abordagem evita a criação de eventos redundantes e mantém a simulação consistente e eficiente. Assim, a lista de eventos contém apenas eventos futuros relevantes, enquanto as alterações internas de estado são tratadas diretamente no momento da ocorrência do evento.

Ainda no ficheiro `"Triagem.py"` está a função `tempo_medio FilaTriagem` que serve para obter dados acerca do tempo médio de espera na fila de triagem. Esta estatística vai ser usada mais à frente na interface de simulação:

```
def tempo_medio_FilaTriagem(doentes_atendidos):
    diferenca = []
    for doente in doentes_atendidos:
        if doente["tentrada_triagem"] != None:
            diferenca.append((doente["tentrada_triagem"] - doente["tchegada"]))

    media = round(sum(diferenca)/len(diferenca),2)

    return media
```

3.3 Consultorios.py

Após os pacientes serem atendidos na receção, estes necessitam de ser encaminhados para as respetivas consultas médicas. Para tal, os pacientes são processados e inseridos nas filas de espera das consultas correspondentes à sua especialidade. Toda a lógica associada à gestão dos consultórios encontra-se implementada no ficheiro `Consultorios.py`, o qual importa as bibliotecas **JSON** e **numpy**.

Antes de detalhar o funcionamento deste ficheiro, torna-se fundamental analisar a base de dados de médicos da clínica. Com o objetivo de conferir maior realismo à simulação, foi criado um ficheiro JSON que contém médicos das **11 especialidades definidas**, distribuídos de acordo com a frequência e relevância de cada especialidade na clínica.

Na base de dados, cada médico é definido do seguinte modo:

```
{"nome":"Helena Santos",  
"id":"m1",  
"ocupado":false,  
"doente":null,  
"inicio_consulta":0.0,  
"fim_consulta":0.0,  
"especialidade":"Cardiologia",  
"inicio_turno":0,  
"fim_turno":430}
```

Através da análise desta estrutura, é possível observar que os médicos apresentam turnos distintos. Cada médico trabalha um total de 8 horas por dia, sendo que, nas especialidades com mais do que um médico, os turnos encontram-se intercalados de forma a garantir uma maior cobertura horária. Por outro lado, as especialidades que dispõem apenas de um médico funcionam exclusivamente durante um turno de 8 horas, que pode corresponder ao período inicial ou final do dia.

Face a esta organização, surgiu a necessidade de desenvolver uma função responsável por carregar os médicos a partir do ficheiro JSON e inicializar os seus dados para a simulação.

```
def carregaMedicos(ficheiro):  
    medicos = []  
    with open("./tentativa2/medicos.json", "r", encoding="utf-8") as p:  
        lista_medicos = json.load(p)  
        for pessoa in lista_medicos:  
            medico = {  
                "nome": pessoa["nome"],  
                "id": pessoa["id"],  
                "disponibilidade": True,  
                "doente": None,  
                "inicio_consulta":None,  
                "fim_consulta":None,  
                "inicio_turno":pessoa["inicio_turno"],  
                "fim_turno":pessoa["fim_turno"],
```

```
        "especialidade": pessoa["especialidade"],
        "ndoentes_atendidos": 0,
        "id_doentes_atendidos" : [],
        "tempo_ocupado":0
    }
    medicos.append(medico)

return medicos
```

Esta função abre o ficheiro medicos.json e, para cada médico presente na base de dados, cria um dicionário que contém tanto informações pessoais como variáveis de estado necessárias à simulação. São inicializados os atributos relacionados com a disponibilidade do médico, as consultas realizadas e as métricas de desempenho. Por fim, todos os médicos são adicionados a uma lista, que é devolvida pela função.

Uma vez que determinadas especialidades estão associadas a mais do que um médico, a clínica foi organizada em secções, sendo que cada secção corresponde a uma especialidade médica. Dentro de cada secção existem vários consultórios, e cada consultório está associado a um médico específico.

```
seccoes = [{"especialidade": "Dermatologia", "id": "s1", "consultorios": []},

            {"especialidade": "Gastroenterologia", "id": "s2", "consultorios": []},

            {"especialidade": "Pneumonologia", "id": "s3", "consultorios": []},

            {"especialidade": "Cardiologia", "id": "s4", "consultorios": []},

            {"especialidade": "Endocrinologia", "id": "s5", "consultorios": []},

            {"especialidade": "Ortopedia", "id": "s6", "consultorios": []},

            {"especialidade": "Neurologia", "id": "s7", "consultorios": []},

            {"especialidade": "Ginecologia e Obstetrícia", "id": "s8", "consultorios": []},
```

```

{"especialidade": "Psiquiatria", "id": "s9","consultorios":[]},

{"especialidade": "Pediatria", "id": "s10","consultorios":[]},

{"especialidade": "Medicina Geral", "id": "s11","consultorios":[]}]

```

Inicialmente, a lista de consultórios de cada secção encontra-se vazia. Assim, foi necessário desenvolver uma função responsável pela criação dos consultórios e pela associação correta entre médicos e especialidades.

```

def cria_id(medico):
    c = medico["id"]
    num = c[1:]
    return num

def criaConsultorios(medicos,seccoes):
    for medico in medicos:
        n = 0
        while n < len(seccoes):
            if medico["especialidade"] == seccoes[n]["especialidade"]:
                seccoes[n]["consultorios"].append({"id":"c" + cria_id(medico), "medico":medico})
                n +=1
            else:
                n +=1
        return seccoes

```

A função `criaConsultorios` percorre a lista de médicos e, para cada um, identifica a secção correspondente à sua especialidade. Em seguida, é criado um consultório com um identificador próprio e este é associado ao médico em questão.

Para garantir coerência entre os identificadores dos médicos e dos consultórios, foi criada a função auxiliar `cria id`, que extrai a parte numérica do identificador do médico (por exemplo, "m3- "3"), permitindo que o consultório associado utilize o mesmo número.

Com os consultórios criados e com os médicos carregados, é necessário associá-los à simulação.

Primeiramente, a função `encontraDoente` tem o objetivo de remover um doente da fila de espera quando este já entrou no consultório, garantindo a coerência entre o estado do doente e as filas.

```
def encontraDoente(doente, fila):
    if doente != None:
        v = 0
        pronto = False
        while v < len(fila) and not pronto:
            if fila[v]["id"] == doente["id"] and doente["tentrada_consultorio"] != None:
                fila.pop(v)
                pronto = True
            else:
                v += 1
        return fila
```

A função recebe um doente e uma fila. Caso o doente exista, percorre a fila com o mesmo id e verifica se o doente já entrou no consultório se este já tiver tempo de entrada no consultório. Quando estas condições são satisfeitas, o doente é removido da fila utilizando o método `pop` e é retornada a fila agora sem esse doente.

Por outro lado, a função `atribuir_doente_a_medico` é responsável por atribuir um doente a um médico disponível, respeitando a especialidade médica e o horário de trabalho do médico.

```
def atribuir_doente_a_medico(doente, seccoes, t_atual):
    medicos_seccao = None
    i = 0

    while i < len(seccoes) and medicos_seccao is None:
        sec = seccoes[i]
        if sec["especialidade"] == doente["especialidade"]:
```

```

        medicos_seccao = [c["medico"] for c in sec["consultorios"]]
        i += 1

for m in medicos_seccao:
    m["disponibilidade"] = (m["inicio_turno"] <= t_atual < m["fim_turno"] and m["d

disponiveis = [m for m in medicos_seccao if m["inicio_turno"] <= t_atual < m["fim

if not disponiveis:
    return None

medico = min(disponiveis, key=lambda m: m["ndoentes_atendidos"])

tempo = None
i = 0

while i < len(tempo_consulta_especialidade) and tempo is None:
    t = tempo_consulta_especialidade[i]
    if t["especialidade"] == medico["especialidade"]:
        tempo = max(5, np.random.normal(t["tempo_medio_consulta"], 5))
    i += 1

medico["doente"] = doente
medico["disponibilidade"] = False
medico["inicio_consulta"] = round(t_atual, 2)
medico["fim_consulta"] = round(t_atual + tempo, 2)
doente["tentrada_consultorio"] = medico["inicio_consulta"]
doente["tsaida_consultorio"] = medico["fim_consulta"]
medico["ndoentes_atendidos"] += 1
medico["id_doentes_atendidos"].append(doente["id"])
medico["tempo_ocupado"] = round(medico["tempo_ocupado"] + tempo, 2)
doente["estado_final"] = "CONSULTA" #registo

```

```
return medico #encontrou um médico
```

Em primeiro lugar, a função identifica a secção correspondente à especialidade do doente e obtém a lista de médicos dessa secção. De seguida, verifica quais os médicos que se encontram disponíveis no instante atual da simulação, ou seja, cujo turno está ativo e que não se encontram a atender outro doente.

Caso existam médicos disponíveis, é selecionado o médico que atendeu menos doentes até ao momento, promovendo assim uma distribuição mais equilibrada da carga de trabalho.

A duração da consulta é então calculada com base na especialidade médica, recorrendo a uma **distribuição normal** centrada no tempo médio definido para essa especialidade, sendo garantido um tempo mínimo de consulta.

```
tempo_consulta_especialidade = [  
    {"especialidade": "Cardiologia", "tempo_medio_consulta": 30},  
    {"especialidade": "Pediatria", "tempo_medio_consulta": 20},  
    {"especialidade": "Dermatologia", "tempo_medio_consulta": 15},  
    {"especialidade": "Gastroenterologia", "tempo_medio_consulta": 25},  
    {"especialidade": "Pneumonologia", "tempo_medio_consulta": 25},  
    {"especialidade": "Endocrinologia", "tempo_medio_consulta": 20},  
    {"especialidade": "Ortopedia", "tempo_medio_consulta": 20},  
    {"especialidade": "Neurologia", "tempo_medio_consulta": 30},  
    {"especialidade": "Ginecologia e Obstetrícia", "tempo_medio_consulta": 25},  
    {"especialidade": "Psiquiatria", "tempo_medio_consulta": 45},  
    {"especialidade": "Medicina Geral", "tempo_medio_consulta": 15}  
]
```

Por fim, são atualizados os estados do médico e do doente, incluindo os tempos de início e fim da consulta, as estatísticas do médico e o estado final do doente. A função devolve o médico atribuído ou None caso não exista disponibilidade.

Assim como se ocupam os médicos, também é necessário desocupa-los e, para isso, foi desenvolvida a função `desocuparMedico` que garante que após a consulta o dicionário do médico é atualizado e ele consiga atender o próximo paciente.

```
def desocuparMedico(medico,t_atual):  
    if medico != None:  
        if medico["fim_consulta"] != None:  
            if t_atual >= medico["fim_consulta"] :  
  
                doente = medico["doente"]  
                doente["estado_final"] = "ATENDIDO" #registro  
  
                medico.update({  
                    "disponibilidade": True,  
                    "doente": None,  
                    "inicio_consulta":None,  
                    "fim_consulta":None,  
                })  
  
    return medico
```

A função recebe um médico e o instante atual da simulação. Caso o médico se encontre em consulta e o tempo atual seja igual ou superior ao tempo de fim da consulta, considera-se que a consulta terminou.

Nessa situação, o estado final do doente é atualizado para "ATENDIDO", e os atributos do médico relacionados com a consulta são reinicializados, nomeadamente a disponibilidade, o doente associado e os tempos de consulta.

Apesar das funções anteriores gerirem corretamente a atribuição e libertação de médicos, estas não são responsáveis pela organização das filas de espera. Após a triagem os doentes deixam de ter prioridade com base nos critérios iniciais, sendo que quem tem consulta marcada tem prioridade sobre quem não tem consulta. [Ver a função que define se o paciente tem consulta](#)

Neste contexto, surge a função tentar atribuir fila que gere as filas de espera e tenta atribuir vários doentes, respeitando prioridades. Esta função seleciona o doente a ser atendido, dando prioridade aos doentes com consulta marcada, e apenas considera os doentes sem consulta quando a fila prioritária se encontra vazia. Sempre que um doente é atribuído

a um médico, este é removido da fila correspondente e é criado um evento do tipo "SAI CONSULTORIO", que representa o momento em que a consulta termina. Este processo é repetido enquanto existirem doentes na fila e médicos disponíveis.

```
def tentar_atribuir_fila(esp, filas_consultas, seccoes, eventos, t_atual):
    continuar = True

    while continuar == True: #enquanto houverem doentes na fila
        if filas_consultas[esp]["com_consulta"]:
            doente = filas_consultas[esp]["com_consulta"][0]
            origem = "com_consulta"

        elif filas_consultas[esp]["sem_consulta"]:
            doente = filas_consultas[esp]["sem_consulta"][0]
            origem = "sem_consulta"

        else: #as filas estão vazias
            continuar = False

    if continuar == True: #se não estiverem vazias
        medico = atribuir_doente_a_medico(doente, seccoes, t_atual)
        #procura o médico e tenta atribui-lo ao doente

        if medico: #se houver médico disponível
            filas_consultas[esp][origem].pop(0)
            #remove o primeiro paciente que estava na fila dessa especialidade

        eventos.append({
            "tempo": medico["fim_consulta"],
            "tipo": "SAI CONSULTORIO",
            "doente": doente,
            "medico": medico
```

```
}  
  
#atualiza o evento para SAI CONSULTÓRIO  
#que é adicionado aos eventos da simulação  
  
eventos.sort(key=lambda e: e["tempo"])  
#ordena os eventos por ordem cronológica  
  
else: #caso não exista médico disponível  
    continuar = False  
    #a função termina
```

Deste modo, esta função desempenha um papel central na simulação, garantindo que os doentes são atendidos assim que existam médicos disponíveis, atualizando corretamente as filas, aplicando critérios de prioridade e coordenando a criação de eventos temporais. Assim, assegura-se um fluxo contínuo e realista de atendimento dentro da clínica.

A função atribuir doente a medico é responsável pela lógica individual de atribuição de um doente a um médico disponível, enquanto a função tentar atribuir fila atua ao nível da gestão global do sistema, selecionando doentes das filas de espera, aplicando critérios de prioridade e coordenando a criação de eventos na simulação.

Apesar de a função tentar atribuir fila remover explicitamente os doentes das filas no momento em que estes entram em consulta, a função encontraDoente mantém-se relevante como mecanismo auxiliar de consistência, permitindo garantir que um doente não permanece indevidamente em filas de espera após ter iniciado a consulta.

O restante conteúdo do ficheiro relaciona-se com a obtenção de estatísticas para criação de gráficos.

1. Função do tamanho das filas consultas por especialidade: para cada especialidade guarda o tamanho das filas com e sem consulta e calcula o total.

```
def tamanho_filas_consultas(filas_consultas,tam_filas):  
  
    for esp, fila in filas_consultas.items():
```

```

    if esp not in tam_filas:
        tam_filas[esp] = {
            "sem_consulta" : [],
            "com_consulta" : [],
            "tam_total" : []
        }
        tam_filas[esp]["sem_consulta"] = [len(fila["sem_consulta"])]
        tam_filas[esp]["com_consulta"] = [len(fila["com_consulta"])]
        tam_filas[esp]["tam_total"] = [len(fila["sem_consulta"]) + len(fila["com_consulta"])]

    else:
        tam_filas[esp]["sem_consulta"] = tam_filas[esp]["sem_consulta"] + [len(fila["sem_consulta"])]
        tam_filas[esp]["com_consulta"] = tam_filas[esp]["com_consulta"] + [len(fila["com_consulta"])]
        tam_filas[esp]["tam_total"] = tam_filas[esp]["tam_total"] + [len(fila["sem_consulta"]) + len(fila["com_consulta"])]

return tam_filas

```

2. Função do tamanho total das filas consultas: soma o número total de doentes em todas as especialidades.

```

def tamanho_total_filasconsultas(filas_consultas,tam_filas,tam_filas_total):
    soma = 0

    for esp,tams in tam_filas.items():
        soma += tam_filas[esp]["tam_total"][-1]

    tam_filas_total.append(soma)

return tam_filas_total

```

3. Função da média do tamanho das filas consultas: calcula médias das filas com e sem consulta e uma média global por especialidade. Trata de casos em que não há dados.

```
def media_tamanho_filas_consultas(tam_filas):

    med_filas = {}

    def calcula_media(fila):
        media = None
        if len(fila) != 0:
            media = sum(fila)/len(fila)
        return media

    for esp, fila in tam_filas.items():

        med_filas[esp] = {
            "media_sem_consulta": calcula_media(fila.get("sem_consulta", [])),
            "media_com_consulta": calcula_media(fila.get("com_consulta", [])),}
        if med_filas[esp]["media_com_consulta"] == None and med_filas[esp]["media_sem_consulta"] == None:
            med_filas[esp]["media_da_especialidade"] = None
        elif med_filas[esp]["media_com_consulta"] == None:
            med_filas[esp]["media_da_especialidade"] = med_filas[esp]["media_sem_consulta"]
        elif med_filas[esp]["media_sem_consulta"] == None:
            med_filas[esp]["media_da_especialidade"] = med_filas[esp]["media_com_consulta"]
        else:
            med_filas[esp]["media_da_especialidade"] = (med_filas[esp]["media_sem_consulta"] + med_filas[esp]["media_com_consulta"])/2

    return med_filas
```

4. Função da média global do tamanho das filas na clínica: junta as médias válidas de todas as especialidades e calcula a média final.

```
def med_filas_consultas(med_filas):
    lista_soma = []
```

```
for _,media in med_filas.items():

    if media["media_da_especialidade"] != None:
        lista_soma.append(media["media_da_especialidade"])

media_final = None

if lista_soma != []:
    media_final = sum(lista_soma)/len(lista_soma)

return media_final
```

5. Função do tempo médio de espera entre a triagem e a entrada no consultório: para cada doente atendido calcula a diferença entre a entrada no consultório e a saída da triagem. Calcula a média dessas diferenças.

```
def tempo_medio_FilasConsultas(doentes_atendidos):

    diferenca = []
    for doente in doentes_atendidos:
        if doente["tentrada_consultorio"] != None:
            diferenca.append((doente["tentrada_consultorio"] - doente["tsaida_triagem"]))

    media = round( sum(diferenca)/len(diferenca),2)

    return media
```

4 Simulação de eventos

4.1 Lógica de tempos e eventos

A simulação tem como principal objetivo registar e recolher dados com a evolução do tempo e variação dos parâmetros de definição da simulação. Então, a mesma deve seguir algumas regras e limites impostos por estas mesmas condições. Os parâmetros mais básicos da simulação são o tempo atual de simulação, "t atual", e o tempo final de simulação, "t final"(está definido para 12 horas de simulação, 12horasx60minutos).

Para o correto funcionamento da simulação, tivemos de definir os principais acontecimentos numa lista de eventos, para que quando o tempo de simulação se igualasse com o tempo do evento, os comandos do código executassem e os dados se atualizassem. Então, os eventos na nossa simulação podem ser classificados como:

1. "CHEGADA"
2. "ENTRA TRIAGEM"
3. "SAI TRIAGEM"
4. "ENTRA CONSULTORIO"
5. "SAI CONSULTORIO"

Com o decorrer do tempo de simulação, os eventos são retirados da fila de eventos e são processados de forma a executar comandos e funções, responsáveis por tratar e gerar dados. Então, a simulação é controlada por um ciclo while que garante que a simulação decorre entre os limites de tempo impostos e o processamento dos eventos.

```
while eventos and t_atual < tfinal:
    evento = eventos.pop(0)
    t_atual = evento["tempo"]
    doente = evento["doente"]
```

Existem também algumas listas que registam dados e métricas importantíssimas para o tratamento de resultados e conclusões a cada ciclo.

```
tempos = []
    tam_fila_triagem = []
    tam_filas_consultas = {}
    tam_filas_consultas_total = []
    ocupacao_medicos = []
```

A cada ciclo, estes dados são tratados com funções auxiliares, ou simplesmente adicionados à respetiva lista.

```
while eventos and t_atual < tfinal:
    evento = eventos.pop(0)
    t_atual = evento["tempo"]
    doente = evento["doente"]

    #-----REGISTO PARA GRÁFICOS-----

    #tamanho total da fila de triagem
    fila_total_triagem = len(fila_triagem[0]) + len(fila_triagem[1])

    #tamanho total das filas para as consultas
    tam_filas_consultas = con.tamanho_filas_consultas(filas_consultas,tam_filas_consultas)
    tam_filas_consultas_total = con.tamanho_total_filasconsultas(filas_consultas,tam_filas_consultas)

    #médicos ocupados
    medicos_ocupados = sum(1 for m in medicos if m["doente"] is not None)
    taxa_ocupacao = (medicos_ocupados / len(medicos)) *100

    #guardar valores
    tempos.append(t_atual)
    tam_fila_triagem.append(fila_total_triagem)
    #tam_fila_consultas.append(fila_total_consultas)
    ocupacao_medicos.append(taxa_ocupacao)
```

[Ver a função tamanho filas consultas](#) [Ver a função tamanho total filasconsultas](#)

4.2 Evento de Chegada

O primeiro evento da simulação gerado é a chegada do primeiro paciente à clínica, o qual passa por uma validação do tempo de chegada.

```
t_chegada = np.random.exponential(1/taxas[0][2])

validado = False
while not validado:
    i = random.randint(0, len(doentes)-1)
    candidato = doentes[i]

    if ch.chegada_valida(candidato, t_chegada, horarios):

        doente = doentes.pop(i)
        doente["tchegada"] = round(t_chegada,2)
        eventos.append({
            "tempo": round(t_chegada,2),
            "tipo": "CHEGADA",
            "doente": doente
        })
        validado = True
        doentes_atendidos.append(doente) #registro
        doente["estado_final"] = "FILA_TRIAGEM" #registro
```

Nesta validação, são comparados 3 parâmetros, o tempo de chegada apurado para o paciente, a especialidade para a qual este paciente se dirigirá e a janela de funcionamento da especialidade; esta validação garante que o tempo de chegada do doente deve de estar compreendido na janela de funcionamento da especialidade, tomando como princípio que um paciente não daria entrada no consultório algumas horas em antes de ser atendido. Caso o tempo de chegada se enquadre nesta validação, o tempo de chegada é registado no dicionário

do doente e é definido o primeiro evento "CHEGADA" da simulação, caso contrário, serão testados novos pacientes até que o sistema encontre um tempo adequado à validação e possa ser criado o evento.

Todos os eventos criados são adicionados a uma lista de eventos, sendo que cada evento individualmente é um dicionário que contém os parâmetros "tipo", "doente" e "tempo". No caso do evento de "CHEGADA", é o evento responsável por desencadear todos os acontecimentos futuros relativos a esse paciente. Quando o tempo de simulação se iguala ao parâmetro "tempo" do evento de chegada, o evento sai da lista de eventos e o doente é adicionado a uma fila de triagem.

```
if evento["tipo"] == "CHEGADA":  
    #doente = evento["doente"]  
    fila_triagem = tr.chegadaAntesTriagem(doente, fila_triagem)
```

Neste ponto, os eventos "CHEGADA" e "ENTRA TRIAGEM" são condensados numa única condição, porque a chegada do doente implica a entrada do mesmo para a fila de atendimento. Este pequeno detalhe diminui a exaustão do sistema, ao não adicionar eventos desnecessários.

Ainda no seguimento da condição do evento ser do tipo chegada, o paciente tenta ocupar um balcão de atendimento, assim que estiver em primeiro lugar da fila de espera relativa ao tipo de balcão (se prioritário ou não), e quando o mesmo estiver disponível. Caso a ocupação falhe, será dada uma nova tentativa, mas quando o balcão estiver disponível. Isto ocorrerá no desenvolvimento do evento "SAI TRIAGEM". [Ver a nova tentativa de ocupação do balcão no evento "SAI TRIAGEM"](#)

```
fila_triagem, balcoes, eventos_gerados =  
tr.ocupar_balcaoTriagem(fila_triagem, balcoes, t_atual)  
for evento in eventos_gerados:  
    if evento != None:  
        eventos.append(evento)
```

Quando o doente ocupa o balcão, os seus dados do dicionário são atualizados, os parâmetros "tentrada triagem" e "tsaida triagem" obtêm agora um valor que não None. O que

define estes dois parâmetros é o tempo atual de simulação, que é igual ao tempo de entrada no balcão, e o tempo de triagem que segue uma distribuição normal, do qual é possível obter o tempo de desocupação do balcão através da soma do tempo de ocupação do balcão e o próprio tempo de triagem. Se o doente ocupa um balcão, é criado um novo evento do tipo "SAI TRIAGEM", com relação ao mesmo doente e com um tempo igual ao tempo de desocupação de balcão. Este evento também é adicionado à lista de eventos. [Ver a função ocupa balcao](#)

No final da execução da condição do evento "CHEGADA", é criada uma nova chegada de um novo doente. Esta chegada segue a lógica da chegada do primeiro paciente, com a única diferença que o tempo de chegada neste caso é igual ao tempo de simulação atual somado com o tempo de intervalo gerado por uma distribuição exponencial negativa com aproximação a um parâmetro de Poisson.

```
for tax in taxas:
    if t_atual >= tax[0] and t_atual <= tax[1]:
        taxa_atual = tax[2]

    intervalo = np.random.exponential(1 / taxa_atual)
    proxima_chegada = t_atual + intervalo

    if proxima_chegada < tfinal and doentes:

        validado = False
        while not validado:
            j = random.randint(0, len(doentes)-1)
            candidato = doentes[j]

            if ch.chegada_valida(candidato, proxima_chegada, horarios):

                validado = True
                novo_doente = doentes.pop(j)
                novo_doente["tchegada"] = round(proxima_chegada,2)
```

```
eventos.append({
    "tempo": round(proxima_chegada,2),
    "tipo": "CHEGADA",
    "doente": novo_doente
})
doentes_atendidos.append(doente) #registro
doente["estado_final"] = "FILA_TRIAGEM" #registro
```

Esta abordagem torna a lógica de sequenciamento de eventos mais fluida e fiel à realidade, uma vantagem sobre a abordagem de criação de todos os tempos de chegada no início da simulação, o que poderia criar uma sensação de condicionamento artificial e sobrecarregamento do sistema. Depois da validação do novo tempo de chegada do paciente, é criado um novo evento "CHEGADA" que é adicionado à lista de eventos.

4.3 Evento de Saída da Triagem

No caso do evento ser do tipo "SAI TRIAGEM", o primeiro acontecimento é definido pela desocupação dos balcões. O dicionário do balcão é atualizado, aumentando em uma unidade o parâmetro "ndoentes atendidos" e o parâmetro "tempo ocupado" soma-se ao tempo de triagem. O balcão é desocupado e está agora disponível a receber um novo paciente. [Ver a função desocupa balcao](#)

Neste ponto, ocorre a nova tentativa de ocupação de um balcão caso a anterior tenha falhado, já que é neste ponto que é certo que um balcão está desocupado.

```
elif evento["tipo"] == "SAI_TRIAGEM":

    balcoes,_ = tr.desocupar_balcaoTriagem(balcoes, t_atual)
    fila_triagem, balcoes, novos =
    tr.ocupar_balcaoTriagem(fila_triagem, balcoes, t_atual)
```

Também o dicionário do balcão é atualizado, aumentando em uma unidade o parâmetro "ndoentes atendidos" e o parâmetro "tempo ocupado" soma-se ao tempo de triagem. Assim que o tempo de simulação iguala-se ao tempo de saída da triagem do dicionário do paciente,

o balcão é desocupado e está agora disponível a receber um novo paciente. Adicionalmente, é criado um novo evento, mas agora do tipo "SAI TRIAGEM". [Ver a função desocupa balcao](#)

Na última fase do evento, o doente entra na fila específica da especialidade (se tem ou não consulta), e tenta, pela primeira vez, ocupar um consultório. Esta lógica é muito semelhante ao caso da relação entre os eventos "CHEGADA" e "ENTRA TRIAGEM", pois também a saída da triagem impõe a tentativa de entrada num consultório. Logo, a condensação dos eventos "SAI TRIAGEM" e "ENTRA CONSULTORIO" facilitam o desempenho do sistema. Caso a tentativa de entrar no consultório falhe, será dada uma nova tentativa quando um consultório ficar livre. A entrada no consultório, é talvez, o acontecimento mais complexo, e por isso, a explicação será dada mais adiante, na subsecção 4.4.

[Ver explicação sobre a tentativa de ocupação de consultório.](#)

```
if doente["consulta"]:  
    filas_consultas[esp]["com_consulta"].append(doente)  
else:  
    filas_consultas[esp]["sem_consulta"].append(doente)  
  
con.tentar_atribuir_fila(esp, filas_consultas, seccoos, eventos, t_atual)
```

4.4 Evento de Saída do Consultório

Por último, o evento do tipo "SAI CONSULTORIO" tem um parâmetro adicional em relação aos restantes eventos, o parâmetro "medico", que define qual médico irá atender o paciente. Este parâmetro permite facilitar a ocupação do consultório, pois é definido pela função que gera o evento, a função "tentar atribuir fila" ([Ver a função tenta atribuir fila](#)). Esta função calcula qual é o próximo médico a desocupar. Isto permite diminuir o número de eventos que representam as tentativas do paciente entrar no consultório a uma só, cujo tempo do evento é igual ao tempo de fim da consulta corrente do médico; caso contrário, seriam adicionados vários eventos "SAI CONSULTORIO" à lista de eventos em pequenos intervalos de tempo até que o paciente conseguisse entrar no consultório, resultando numa falsa sensação de condicionamento e uma sobrecarga no sistema.

```
{  
"tempo": medico["fim_consulta"],  
"tipo": "SAI_CONSULTORIO",  
"doente": doente,  
"medico": medico  
}
```

O evento começa por desocupar o médico, atualiza o seu dicionário e deixa-o disponível para atender o próximo paciente ([Ver a função desocupa medico](#)). De seguida, tenta ocupar novamente o consultório com tentativas falhadas anteriores com a função "tenta atribuir fila"[Ver a função tenta atribuir fila](#)). Daqui, é determinado qual o próximo médico que será desocupado e devolve o médico, com o auxílio da função "atribuir doente a medico"([Ver a função atribuir doente a medico](#)) ; caso encontre um médico, o resultado é o dicionário do médico, senão retorna o None. Se o médico for um dicionário o doente ocupa o consultório, caso contrário é criado evento "SAI CONSULTORIO". Este segundo caso garante que na primeira tentativa de ocupação do consultório no evento "SAI TRIAGEM" gere o evento "SAI CONSULTORIO" caso a ocupação falhe.

4.5 Resultados e Obtenção de Dados

No fim, a simulação retorna dados de duas formas:

1. return;
2. yield.

O **return** devolve os dados assim que toda a informação é tratada e bem definida; retornando informação essencial para a construção de gráficos e o cálculo de métricas de desempenho.

O **yield** é uma função nova que não abordamos nas aulas, mas que decidimos utilizar porque, como poderemos ver a seguir na secção 5, o yield será fundamental para que seja mais fácil gerar uma interface animada e dinâmica. Basicamente, a maior diferença do yield em relação ao return é tornar a simulação uma função geradora, que retorna as informações de forma gradual, e de certa forma, diminui a sobrecarga sobre o sistema.

5 Interface e Modelo Visual

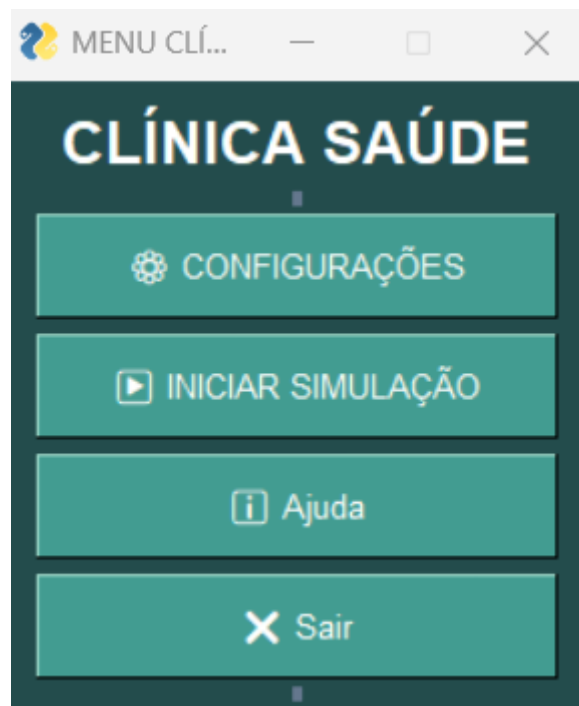
A interface do programa é a peça que permite que o programa seja utilizado de forma mais simples e que qualquer pessoa consiga usar o programa de forma intuitiva. Além disso, permite adicionar elementos visuais que permitem que a informação seja passada de forma mais intuitiva e clara.

Toda a interface foi criada com o auxílio da biblioteca **FreeSimpleGUI**

5.1 Menu

O menu da interface é a primeira janela que aparece assim que se inicia o programa. A partir daqui é possível aceder a todas as funcionalidades do programa:

1. Configurações- permitem alterar alguns parâmetros da simulação;
2. Iniciar Simulação- permite executar a simulação propriamente dita;
3. Ajuda- abre uma janela que explica brevemente a lógica e funcionamento do programa;
4. Sair- fecha a janela do menu e encerra o programa.

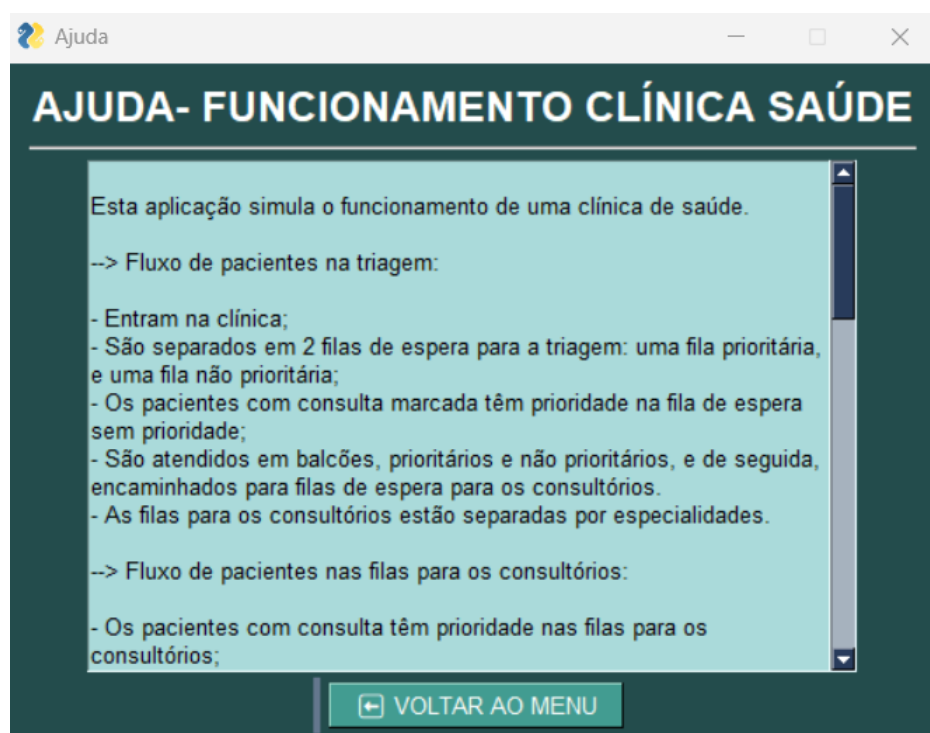


Na verdade, o item menu é a última parte da interface a ficar concluída, já que é necessário que todas as outras funcionalidades estejam previamente prontas para poderem ser adicionadas e vinculadas às funcionalidades dos botões.

O código da interface é relativamente simples. Primeiro, o layout é criado, e o evento gerado pelo clique de cada botão inicia a execução das funções que iniciam as restantes janelas da interface.

5.2 Ajuda

A interface da janela Ajuda mostra um texto que dá um pequeno contexto da trajetória e do fluxo dos doentes da clínica, como é que os diferentes recursos funcionam e alguns dos parâmetros que são registados e que são alteráveis. Adicionalmente, dá instruções e esclarecimentos das funcionalidades e informações das restantes janelas da interface.



Também a janela de Ajuda é bastante simples. A janela continua aberta até que o botão de voltar ao menu seja premido, que desencadeia o fecho da janela, e o Menu é inteligível novamente. A funcionalidade "modal" está definida com esta janela, a qual permite que a janela de Menu continue aberta em simultâneo com a janela de Ajuda, mas não é possível interagir com ela.

5.3 Configurações

A janela de configurações permite alterar os parâmetros da simulação. Os parâmetros alteráveis são:

1. Número total de balcões- é possível variar de 2 a 10;
2. Número de balcões prioritários- é possível variar de 1 a 9;
3. Modo de taxa de chegadas- pode ser (variam por períodos do dia e estão ordenadas de menores para maiores taxas):
 - 1.1 Calmíssimo;
 - 1.2 Calmo;
 - 1.3 Default;
 - 1.4 Cheio;
 - 1.5 Cheíssimo.

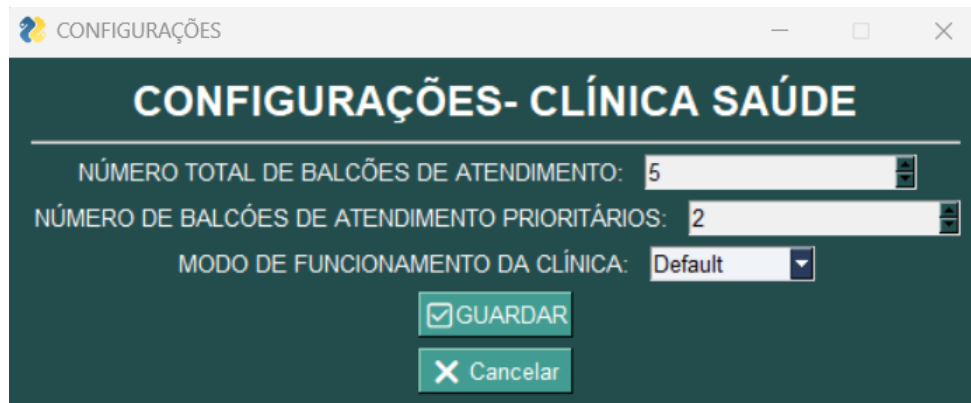
Também existe um botão salvar e um botão cancelar. O botão salvar define os novos parâmetros da simulação caso estes sejam válidos. Casos em que os parâmetros não podem ser salvos são:

1. O número de balcões prioritários é igual ou maior do que o número de balcões totais; evitando que o número de balcões prioritários seja coerente e que ainda hajam balcões não prioritários;
2. O modo de taxas selecionado não exista; em princípio, este erro não deve de ocorrer, pois a seleção dos parâmetros apenas podem ser selecionados e não personalizados.

Se os parâmetros forem inválidos, aparece uma janela de erro, a indicar a razão do erro. Adicionalmente, os parâmetros são restaurados para os parâmetros Default.

O botão cancelar fecha a janela de configurações, e volta para o Menu. Assim como na janela de ajuda, a funcionalidade "modal" também está ativa, deixando a janela de Menu não inteligível.

Também a funcionalidade "finalize" está ativa. O objetivo desta funcionalidade é impedir alterar valores e informações desta janela assim que esta janela seja finalizada.



Para criar a janela de **Configurações**, foram utilizadas novas funcionalidades. Para além das funcionalidades Button e Text, que criam, respectivamente, botões e caixas de texto, também foram utilizadas as funcionalidades Spin e Combo.

A funcionalidade Spin cria uma seleção no estilo de seleção com setas, podendo aumentar ou diminuir uma unidade. Esta funcionalidade é utilizada para definir o número de balcões. A funcionalidade "readonly" está ativada, o que faz com que o usuário não consiga editar manualmente os valores.

A funcionalidade Combo permite a seleção a partir de uma lista. Esta funcionalidade é utilizada para definir o modo taxas.

Na modelação do evento loop, existem validações e alteração de valores quando existe interação com os botões. Para isso, a função que gera a janela de configurações recebe dois parâmetros:

1. Configuração atual- contém todas as definições atuais dos parâmetros alteráveis na simulação (no início da simulação, os valores dos parâmetros são os valores Default). A configuração Default está definida num ficheiro à parte para impedir que a sua informação seja mais fácil de exportar para outros ficheiros e que não seja alterada. Tem a estrutura de um dicionário
2. Modos de taxas- contém a definição dos diferentes modos alteráveis das taxas. Para obter os diferentes valores, os valores Default são multiplicados por um número inteiro. Por exemplo, os valores do modo Cheio obtêm-se multiplicando os valores Default por 5. Os valores das taxas são variáveis ao longo do dia, independentemente do modo das taxas, e os intervalos de variação de taxas são iguais em todos os modos. Tem a estrutura de um dicionário.

Para a validação do número de balcões prioritários, foi usada a condição "nbalcoes <= nbalcoes prior". Caso se verifique esta condição, é utilizada a funcionalidade popup error com a mensagem de invalidação. A condição é seguida pela função "**continue**". "Continue" faz com que o cumprimento da condição na qual está indentada faça com que o restante código da função não seja executado. Assim, impede a execução do código seguinte à condição, a alteração do dicionário da Configuração Atual com os valores seleccionados.

```
if event == "-SAVE-":
    nbalcoes = int(values["-BTOTAL-"])
    nbalcoes_prior = int(values["-BPRIOR-"])
    modo = values["-MOD0-"]

    STOP = True

    if nbalcoes <= nbalcoes_prior:
        sg.popup_error("O número de balcões prioritários
        não pode ser maior que o número total de balcões.")
        continue

        # continue faz com que quando entra uma condição no loop,
        ignora o restante código e inicia um novo loop

    config_atual["nbalcoes"] = nbalcoes
    config_atual["nbalcoes_prior"] = nbalcoes_prior
    config_atual["modo"] = modo
```

Já a validação da seleção do modo é controlada por uma lista que contém todos os modos disponíveis. Caso o modo seleccionado não esteja nessa lista, abre-se uma janela de popup error com a mensagem de invalidação. Também aqui se utiliza "continue".

```
if modo not in modos_taxas:
    sg.popup_error(f"Modo inválido seleccionado: {modo}")
```

continue

```
config_atual["taxas"] = modos_taxas[modo]
```

```
sg.popup("Configurações guardadas com sucesso!"
, title="Sucesso")
```

5.4 Iniciar Simulação

Após a atualização das configurações no menu e a execução do botão de início de simulação, o utilizador é apresentado a uma janela que já inicia a simulação.



Esta estrutura é obtida devido à interface desenvolvida num ficheiro intitulado "teste de interface". Esse ficheiro Python é responsável pela gestão da interface gráfica e pelo controlo da execução da simulação da clínica. Não contém a lógica principal da simulação, mas atua como camada de visualização, interação e monitorização dos estados internos do sistema, funcionando como uma camada intermédia entre o modelo e o utilizador final.

Também para a construção da interface gráfica da simulação foi utilizada a biblioteca FreeSimpleGUI, que permite criar aplicações gráficas de forma simples e modular.

```
import FreeSimpleGUI as sg
import time
```

```
import App2
import Triagem
import Consultorios as con
import Graficos as gr
```

Logo no início do ficheiro são definidas cores e fontes utilizadas em toda a aplicação. Esta abordagem permite garantir consistência visual e facilita a identificação do estado dos diferentes elementos do sistema.

```
PRIMARY = "#429c91"
OCUPADO = "#d15050"
LIVRE = PRIMARY
CADEIRA_LIVRE = "#9aa3ad"
CADEIRA_OCUPADA = "red"
```

As cores são utilizadas, por exemplo, para distinguir elementos livres, ocupados ou indisponíveis, tornando a simulação mais intuitiva para o utilizador.

5.4.1 Balcões

A função principal do módulo é simulação principal, que recebe a configuração atual da simulação e inicializa os elementos gráficos e lógicos necessários.

```
def simulacao_principal(config_atual):
    balcoes = Triagem.criaBalcoes(config_atual)
```

Nesta fase, os balcões de triagem são criados com base nos parâmetros definidos na configuração inicial, no menu, assegurando a coerência entre o modelo e a interface.

Para permitir a interação entre os botões da interface gráfica e as estruturas de dados internas, é criado um mapa de correspondência entre os balcões e as respetivas keys gráficas.

```
def define_Mapa_Balcoes(balcoes):
    MAPA_BALCOES = {}
    ...
    MAPA_BALCOES[key] = ind
```

Este mapeamento permite identificar corretamente qual o balcão selecionado quando ocorre um evento de clique, garantindo uma ligação direta entre a interface e o estado da simulação.

A interface gráfica é organizada em diferentes áreas funcionais. Os balcões de triagem são criados dinamicamente, distinguindo balcões prioritários de não prioritários.

```
def criaFrame_balcoes(balcoes, MAPA_BALCOES):  
    lista_botoes.append(  
        sg.Button("Balcão", key=key, button_color=("white", PRIMARY))  
    )
```

Cada balcão é representado por um botão cuja cor é atualizada em tempo real, refletindo se o balcão se encontra livre ou ocupado.

A atualização em tempo real da cor dos balcões é realizada dentro do ciclo principal da simulação, sempre que o estado do sistema é avançado.

```
# -----  
# ATUALIZAR BALCÕES (CORES)  
# -----  
for key, idx in MAPA_BALCOES.items():  
    balc = balcoes[idx]  
    cor = LIVRE if balc["disponivel"] else OCUPADO  
    window[key].update(button_color=("white", cor))
```

MAPA BALCOES estabelece a correspondência entre cada botão da interface gráfica e o balcão respectivo no modelo. O campo `balc["disponivel"]` indica se o balcão se encontra livre ou ocupado. Consoante esse estado, é selecionada a cor apropriada (LIVRE ou OCUPADO), inicialmente definida. O método `update` altera dinamicamente a cor do botão durante a execução da simulação.

Devido a esta configuração, o utilizador consegue rapidamente perceber se o balcão está a ser ocupado ou não em tempo "real" de simulação.

5.4.2 Cadeiras

A sala de espera é representada graficamente através de cadeiras, cujo número ocupado varia consoante o tamanho da fila de triagem.

```
def cadeiras(linhas=4, colunas=6):  
    cadeira = sg.Text(  
        "", background_color=CADEIRA_LIVRE  
    )  
    ...
```

À medida que o número de doentes em espera aumenta, as cadeiras passam do estado livre para ocupado, permitindo uma visualização imediata da pressão sobre o sistema. No ficheiro, o bloco de código que altera a cor das cadeiras da sala de espera encontra-se dentro do ciclo principal da simulação, depois de calcular o número de doentes em espera:

```
# -----  
# ATUALIZAR SALA DE ESPERA  
# -----  
n_espera = len(fila_triagem[0]) + len(fila_triagem[1])  
  
for i, cadeira in enumerate(CADEIRAS):  
    if i < n_espera:  
        cadeira.update(background_color=OCUPADO)  
    else:  
        cadeira.update(background_color=CADEIRA_LIVRE)
```

Cada cadeira muda de cor de acordo com a sua ocupação: se há um doente na posição correspondente, fica vermelha (OCUPADO), caso contrário mantém-se cinzenta (CADEIRA LIVRE). Este mecanismo permite visualizar em tempo real a ocupação da sala de espera durante a simulação.

5.4.3 Consultórios

Os consultórios são representados na interface gráfica por botões que indicam o estado de cada especialidade e do respectivo médico. Tal como acontece com os balcões e a sala de

espera, a cor dos botões muda dinamicamente para refletir a disponibilidade.

O estado de cada consultório depende de 2 fatores principais:

1. Turno do médico: se o médico está em serviço ou não
2. Ocupação do médico: se o médico está ocupado ou não

Esses parâmetros são analisados pelas seguintes funções:

```
def medico_em_turno(med, t_atual):  
    inicio = med.get("inicio_turno", 0)  
    fim = med.get("fim_turno", 720)  
    return inicio <= t_atual < fim  
  
def em_consulta(med, t_atual):  
    return medico_em_turno(med, t_atual) and med.get("doente") is not None
```

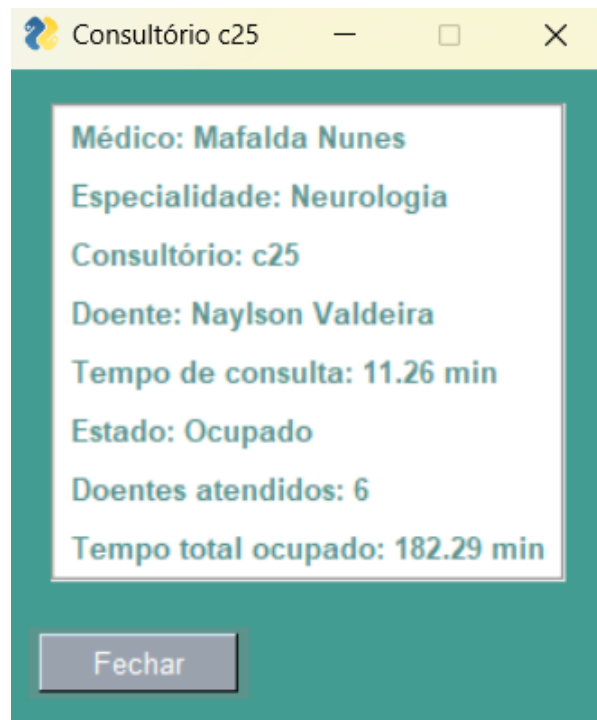
Com base neste estado, a cor do botão é definida:

```
def cor_consultorio(med, t_atual):  
    if em_consulta(med, t_atual):  
        return OCUPADO  
    elif not medico_em_turno(med, t_atual):  
        return "#9aa3ad" # fora de turno  
    else:  
        return LIVRE
```

Dentro do ciclo principal da simulação, todos os consultórios são percorridos e o botão correspondente na interface é atualizado com a cor adequada:

```
for sec_id, lista_consultorios in MAPA_CONSULTORIOS.items():  
    todos_em_consulta = all(em_consulta(cons["medico"], t_atual) for cons in lista_consultorios)  
    cor = OCUPADO if todos_em_consulta else LIVRE  
    key = f"-CONS-{sec_id}-"  
    if key in window.AllKeysDict:  
        window[key].update(button_color=("white", cor))
```

Quando o utilizador clica num consultório, surge uma janela popup com informações detalhadas sobre os médicos e doentes naquele consultório:



Cada atualização dos consultórios depende do tempo da simulação (t atual). A cor dos botões é alterada de acordo com o estado do médico e do consultório de maneira instantânea em que se todos os consultórios da especialidade estiverem ocupados, a unidade fica vermelha. Dentro de cada unidade, se o médico de cada consultório estiver ocupado, o consultório aparece a vermelho e se estiver fora de turno, aparece cinzento até que o t atual coincida com o início de turno do mesmo.

5.4.4 Botões de tempo de espera

Na interface, existem dois botões específicos que permitem ao utilizador obter informação estatística instantânea sobre as filas:

1. Tempo médio de fila de triagem (-TWAIT-) Quando clicado, a simulação é temporariamente pausada para que o utilizador possa visualizar a informação sem que o estado da simulação avance.

O cálculo é feito através da função `Triagem.tempo medio FilaTriagem`, que percorre os doentes atendidos até ao momento e calcula o tempo médio em minutos:

```

if event == "-TWAIT-":
    simulacao_pausada = True
    tempo_medio = Triagem.tempo_medio_FilaTriagem(doentes_atendidos)
    sg.popup(f"Tempo médio de espera na fila para a triagem: {tempo_medio:.2f} mi
    simulacao_pausada = False

```

2. Tempo médio de fila para a consulta (-CWAIT-) Funciona de forma idêntica ao TWAIT, pausando a simulação, calculando o tempo médio e mostrando-o num popup:

```

if event == "-CWAIT-":
    simulacao_pausada = True
    tempo_medio = con.tempo_medio_FilasConsultas(doentes_atendidos)
    sg.popup(f"Tempo médio de espera na fila para a consulta: {tempo_medio:.2f} m
    simulacao_pausada = False

```

Se o utilizador interagir com os botões em diferentes fases da simulação, o resultado para cada um vai variar visto que digere informações em "tempo real".

5.4.5 Tempo

Relativamente ao tempo de simulação, a interface consegue controlar o tempo através de um relógio que avança a cada iteração da simulação. A gestão do mesmo é um dos elementos centrais do funcionamento do sistema. O tempo não avança de forma continua, mas sim de forma discreta, controlada por um gerador, permitindo sincronizar a evolução do modelo com a interface gráfica.

O tempo da simulação é representado por uma variável inteira (t atual), que corresponde ao número de minutos decorridos desde o início da simulação. O valor inicial corresponde ao horário de abertura da clínica.

A cada iteração do gerador da simulação, este valor é incrementado e devolvido à interface:

```
t_atual, fila_triagem, ..., seccoes, doentes_atendidos = next(simulador)
```

Desta forma, `t_atual` funciona como o relógio interno de toda a simulação, sendo utilizado por vários módulos para tomar decisões.

Embora internamente o tempo seja tratado em minutos, é necessário apresentá-lo ao utilizador de forma compreensível. Para esse efeito, é utilizada a função `formatar tempo`, que converte o número de minutos desde o início da simulação para o formato horas:minutos.

```
def formatar_tempo(minutos_desde_inicio, hora_inicio=9, minuto_inicio=0):
    total_minutos = hora_inicio * 60 + minuto_inicio + minutos_desde_inicio
    h = int(total_minutos // 60)
    m = int(total_minutos % 60)
    return f"{h:02d}:{m:02d}"
```

Este valor é depois apresentado na interface gráfica e atualizado a cada iteração:

```
window["-TEMPO-"].update(f"Relógio: {formatar_tempo(t_atual)}")
```

O avanço do tempo ocorre dentro do ciclo de eventos da interface gráfica. Sempre que o botão que pausa a simulação é clicado, a simulação é interrompida:

```
if event == "-PAUSA-":
    simulacao_pausada = not simulacao_pausada
```

Quando pausada a simulação, o gerador não avança, o tempo interno (`t_atual`) mantém-se parado e a interface continua responsiva, permitindo a interação com o utilizador e a consulta de informação.

Porém, quando não se encontra em pausa, é efetuada uma chamada ao gerador:

```
if not simulacao_pausada:
    t_atual, fila_triagem, filas_consultas, ... = next(simulador)
```

Cada chamada a `next(simulador)` corresponde a um avanço temporal da simulação, tipicamente um minuto. Esta unidade temporal é definida pela forma como o tempo é utilizado e interpretado ao longo do modelo, nomeadamente na conversão para horas:minutos, na definição dos turnos dos médicos e no cálculo das métricas de desempenho, todas expressas em

minutos. Este mecanismo garante que o tempo só avança quando a interface está pronta a processar o novo estado.

No `App2.simulação(config atual)` cada `yield` devolve o estado de simulação num determinado instante e o estado interno do gerador fica suspenso até à próxima chamada a `next()`. Por outras palavras, invocar o `next()` é como dizer ao `App2.simulação()` para se executar mais uma vez até ao `yield` e retornar os estados de tempo e disponibilidade no novo instante.

Para além de ser possível pausar e despausar a simulação, é também possível interagir com um botão que acelera a simulação até ao fim. Deste modo, a pessoa consegue terminar a simulação instantaneamente e aceder aos gráficos gerados com os dados obtidos ao longo da mesma.

```
if event == "-FAST-":
    simulacao_pausada = True
    try:
        while True:
            t_atual, fila_triagem,..., seccoes, doentes_atendidos = next(simulador)
    except StopIteration as e:
        resultados_finais = e.value
        sg.popup("Simulação Concluída")
        gr.mostrar_graficos_finais(resultados_finais)
        break
```

6 Resultados e Representações Gráficas

A representação gráfica dos resultados ajuda bastante na compreensão dos dados e métricas registadas, pois é possível interpretar facilmente o funcionamento da clínica.

No nosso programa, os gráficos surgem no final da simulação, assim que todos os dados e métricas estejam completas. Desta forma, conseguimos fazer uma análise completa da evolução das condições ao longo do tempo.

Os gráficos fundamentais para uma análise e compreensão básica do que ocorre durante a simulação são:

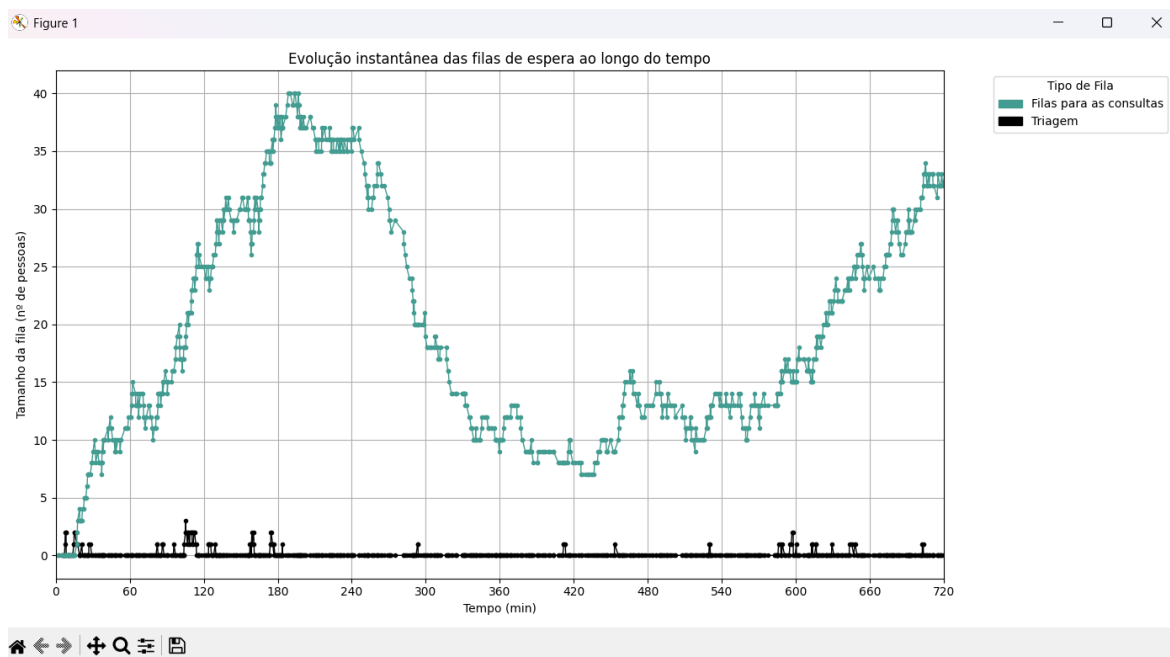
1. Evolução do tamanho das filas de espera em função do tempo;
2. Evolução da taxa de ocupação dos médicos em função do tempo;
3. Gráfico da evolução do tamanho das filas de espera em função do tempo

Com esta informação, é possível retirar conclusões relativas ao comportamento e reação dos recursos disponíveis ao stress submetido e à quantidade de pessoas que chegam à clínica.

6.1 Tamanho das Filas VS Tempo

Este gráfico, em particular, pode ajudar a perceber em que ponto do dia os diferentes órgãos da clínica ficam sobrecarregados e não conseguem vencer a chegada de doentes à clínica. Assim que uma fila de espera cresce de forma significativa e de forma incomum, significa que os recursos estão sobrelotados. A partir daqui, muitas outras métricas tendem a crescer bastante; métricas como taxas de ocupação, tempos médio de espera e o número de doentes atendidos.

De seguida, está um exemplo de um possível gráfico, com as configurações Default (5 balcões, dos quais 2 são prioritários e modos de chegada Default) que representa a evolução do tamanho das filas de espera para a triagem e para as consultas:



Pelo que é possível observar no gráfico, a evolução do tamanho da fila de triagem não mostra grande preocupação quanto à sobrelotação dos balcões, o que significa que para uma afluência de cerca de 330 pacientes, os balcões conseguem atender facilmente todos os pacientes.

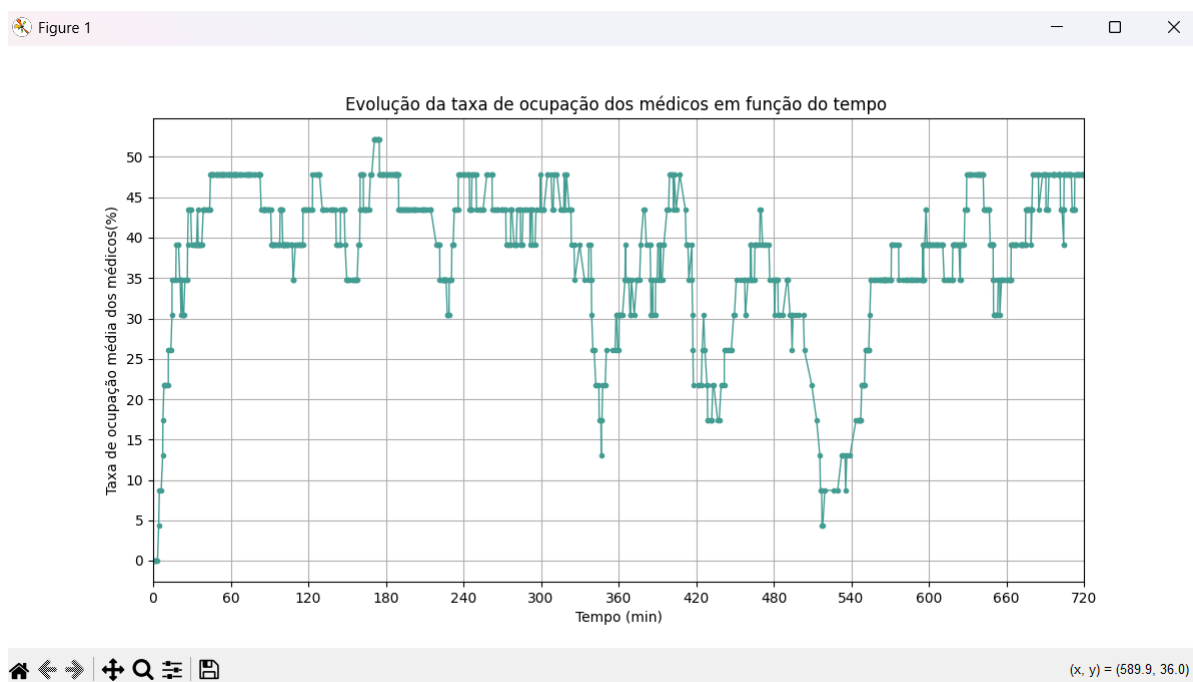
Já a fila de espera para as consultas atinge picos significativos durante o dia. Ainda na primeira hora de funcionamento, o tamanho das filas atinge um pico local, seguido por um crescimento exponencial até à terceira hora de funcionamento. Isto ocorre graças à grande afluência de pacientes que chega nas 2 horas iniciais de simulação, e apesar da quantidade de pessoas chegadas à clínica diminuírem a partir da segunda hora, a sobrelotação dos médicos faz com que as filas tenham um acúmulo de pessoas e atinjam grandes valores de tamanho. Apenas a partir da quarta hora de simulação é que a equipa de médicos consegue atender grande parte das pessoas, pois a taxa de chegada de pessoas é bem menor. Entre a quinta hora de funcionamento e a décima, a clínica tem um período mais calmo, em que os valores do tamanho da fila parecem flutuar à volta das 13 pessoas. Pelos valores serem aproximadamente estáveis, os médicos devem conseguir atender a quantidade de pessoas que chega durante esse período, mas pelo valor médio ser de cerca de 13 pessoas, muito provavelmente, a grande afluência de horas anteriores faz com que haja um acúmulo na fila. Assim conclui-se que neste período os médicos satisfazem o número de chegada de doentes, mas não o acúmulo e o grande número de pessoas que chegou durante horas anteriores. Finalmente, no fim do dia

há outro grande pico e as filas crescem exponencialmente mais uma vez e ajuda a comprovar que no período que antecede este, os médicos já estavam com uma ocupação quase máxima.

6.2 Ocupação dos Médicos VS Tempo

A ocupação dos médicos revela a resposta dos consultórios e da equipa de médicos ao stress de chegadas de pessoas. Se os médicos estiverem a uma taxa próxima ao máximo, significa que os consultórios estão sobrelotados e é esperado um aumento no tamanho da fila para as consultas.

De seguida está um exemplo de um gráfico que mostra a evolução da taxa de ocupação dos médicos em função do tempo.



Para perceber melhor este gráfico, é preciso conhecer os turnos dos médicos. No total são 23 médicos que trabalham durante 8 horas. Para facilitar a compreensão de resultados e a simulação em si, os turnos foram simplificados: cerca de metade dos médicos trabalham as primeiro 8 horas e a outra metade as últimas 8 horas. Isto faz com que haja uma sobreposição de turnos nas 4 horas a metade do dia, ou seja, a partir da quarta hora de funcionamento até à oitava hora todos os médicos estão em funções.

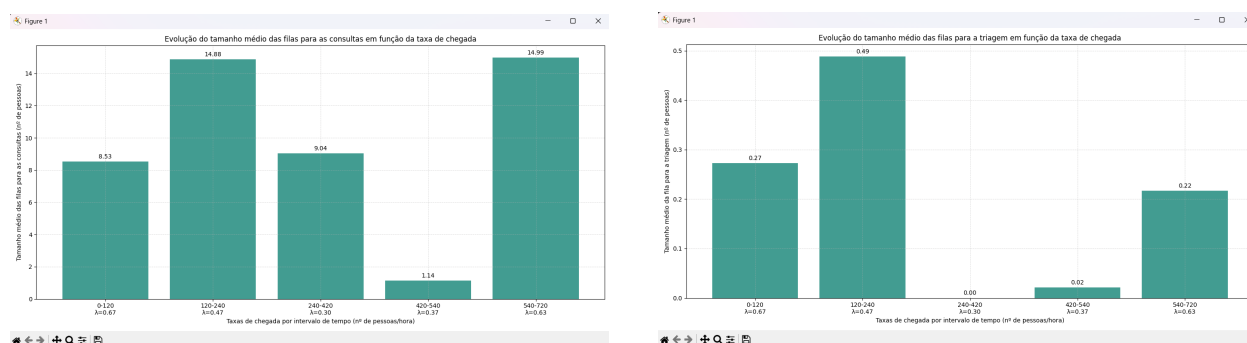
Podemos verificar que na maioria do dia, as taxas de ocupação rondam os 50 por cento. No entanto, este valor tem significados diferentes ao longo do dia. Nas primeiras 4 horas

de funcionamento, 50 por cento da taxa de ocupação significa que quase a totalidade dos médicos em funções está ocupada, já que apenas cerca de metade estão no turno. O mesmo raciocínio aplica-se a partir da oitava hora de funcionamento. Por volta da quinta hora de funcionamento, existe um grande decréscimo na ocupação dos médicos, provavelmente porque não só a taxa de chegada de pessoas é relativamente baixa, mas porque como existe um maior número de médicos em funções, houve um maior número de doentes atendidos entre a quarta e a quinta horas, diminuindo as pessoas na fila de espera. Também há um grande decréscimo a partir da oitava hora, que é a hora em que uma metade dos médicos terminam funções. A partir desta hora, doentes que sejam relativos a especialidades que terminam a essa hora já não dão entrada na clínica, e os pacientes que estavam a ser atendidos por esse grupos de médicos acabam a respetiva consulta e acabam por desocupar o médico. Isto faz com que a taxa de ocupação atinja valores mínimos.

6.3 Tamanho das Filas de Espera VS Taxas de Chegada

Esta informação ajuda a corroborar as conclusões retiradas nos [gráficos que relacionam os tamanhos de fila de espera com o tempo](#) e complementam todas as outras informações retiradas da simulação.

De seguida está um exemplo de um possível gráfico que relaciona o tamanho da fila de triagem com a taxa de chegada de pacientes e outro que relaciona o tamanho da fila para as consultas com a taxa de chegada de pacientes.



Conclui-se através da análise dos gráficos que tanto a triagem como o consultório têm filas maiores em horas a seguir ao horário de pico. Isto ocorre porque apesar do stress de chegadas ser maior no horário de pico, o acúmulo de pessoas é crescente ao longo do dia caso a equipa de médicos não consiga vencer a chegada de pacientes.

Na triagem, observa-se que os valores médios das filas são quase nulos, o que significa que os balões satisfazem eficazmente a chegada de doentes.

No entanto, as filas para as consultas têm valores já significativos. Apesar do número de consultórios ser bem maior em relação aos balcões, as consultas demoram bem mais tempo, o que faz com que a taxa de entrada de pessoas por hora seja bem menor do que a taxa de entrada de pessoas nos balcões. Assim, é natural que o tamanho médio das filas para as consultas seja bem maior.

6.4 Restante Informação

No final da simulação incluímos mais gráficos para além destes que retratam outros parâmetros de uma forma diferente e mais intuitiva. No entanto, todos os resultados que são obtidos pela análise de gráficos anteriores são os mesmos que obtemos nestes gráficos adicionais.

7 Conclusões

O projeto revelou-se particularmente útil, uma vez que permitiu desenvolver novas competências técnicas, nomeadamente na programação e gestão de interfaces gráficas, ao mesmo tempo que fomentou o raciocínio lógico e a capacidade de planear e organizar o código de forma estruturada. Para além disso, surgiu como um desafio que pôs à prova as nossas capacidades de trabalhar sobre pressão e de resolver problemas, visto que estes surgiram muitas vezes e necessitavam de uma resposta rápida para não condicionar o desenvolvimento do projeto.

Este processo de desenvolvimento permitiu-nos também compreender melhor a importância de uma arquitetura de código modular, da documentação clara e da manutenção da coerência entre a interface gráfica e a lógica subjacente da simulação. De maneira geral, o projeto que desenvolvemos é motivo de orgulho porque acreditamos que se apresenta como uma ferramenta interessante e intuitiva do ponto de vista do utilizador.

Por fim, este trabalho evidenciou a relevância da simulação como instrumento pedagógico, permitindo não só analisar fluxos e comportamentos em contexto controlado, como também desenvolver competências transversais, tais como a organização, a gestão de tempo

e a capacidade de colaboração, que serão úteis em projetos futuros e no exercício profissional.